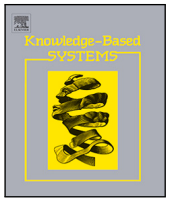
Knowledge-Based Systems 300 (2024) 112144



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





A realistic model extraction attack against graph neural networks

Faqian Guan [a], Tianqing Zhu [b,*], Hanjin Tong [a], Wanlei Zhou [b]

[a] School of Computer Science, China University of Geosciences (Wuhan), Wuhan, 430074, Hubei, China
[b] Faculty of Data Science, City University of Macau, 999078, Macao Special Administrative Region of China

ARTICLE INFO

Keywords:
Graph neural networks
Model extraction attack
Incorrect labels
Fewer queries
Black-box

ABSTRACT

Model extraction attacks are considered to be a significant avenue of vulnerability in machine learning. In model extraction attacks, the attacker repeatedly queries a victim model so as to train a surrogate model that can mimic the output of the victim model. Graph neural networks (GNNs), which are designed to process graph data, were previously thought to be less sensitive to such attacks. This is because, in black-box settings, attackers only have limited access to the victim model. Also, the number of queries any one user can make within a given time window is usually restricted and within this finite number of responses some of the information may contain errors. Yet training a useful surrogate model not only requires a substantial number of queries, but incorrect node labels appearing in the victim GNN's responses is highly problematic. However, in this paper, we demonstrate that GNNs may have a similar vulnerability to model extraction attacks as a normal machine learning model. Our proposed method of extraction addresses the issue of incorrect node labels while also significantly reducing the number of required queries required to train a well-performing model. With this method, GNN extraction attacks are actually highly practical in the real world. Specifically, our proposed methodology incorporates an edge prediction module that introduces potential edges into the original graph data. This module links incorrectly labeled nodes with more accurately labeled ones, thereby mitigating the impact of incorrect labels. And, by increasing the number of possible edges, our approach enables the surrogate model to better leverage the graph's structure, enhancing the contribution of the labeled nodes and allowing the model extraction attack to be executed with fewer queries. Our experiments demonstrate a significant performance improvement over existing approaches, especially in a black-box setting. As such, this research shows that GNNs are also vulnerable to model extraction attacks in real-world scenarios.

1. Introduction

Many real-world datasets can be represented as graphs, including social networks [1–3] and recommendation systems [4–6]. Hence, graph neural networks (GNNs) [7] have been extensively used in machine learning for classification and prediction with graph data [8–10], where the graph data includes both node features and topological structures represent the relationships between nodes. Machine learning significantly enhances our lives but also introduces risks such as adversarial attacks [11–13] and privacy attacks [14–16]. Among these risks, a prevalent threat is the model extraction attack, where attackers train a surrogate model to mimic a target model [14]. Such model extraction attacks risk knowledge theft and can undermine the commercial value of proprietary models. However, once thought to be relatively safe from such attacks, we demonstrate in this paper that GNNs are just as vulnerable to model extraction attacks as traditional machine learning models.

Preliminary research has explored many aspects of GNN model extraction attacks [17–19]. For example, DeFazio et al. [17] formalized one instance of such an attack and presented initial solutions. Shen et al. [19] explored model extraction attacks through various application programming interfaces (APIs), while Wu et al. [18] examined GNN model extraction attacks under the constraints of limited attacker knowledge. However, while previous research on GNN model extraction attacks has shown progress, numerous challenges still remain. These challenges (incorrectly) make current researchers believe that GNNs are relatively able to resist model extraction attacks in a black-box setting. The thinking and challenges are as follows:

First, the target models are imperfect, and their predictions can be erroneous, and these errors in the target model's responses may lead to incorrect decisions. In model extraction attacks, such inaccurate answers may introduce additional uncertainty, complicating the extraction process. Furthermore, in GNNs, incorrect labels not only impact the current node but also neighboring nodes. This propagation of inaccuracies through interconnected nodes can degrade the overall prediction

* Corresponding author.
E-mail address: tianqing.zhu@ieee.org (T. Zhu).

https://doi.org/10.1016/j.knosys.2024.112144
Received 6 February 2024; Received in revised form 12 June 2024; Accepted 14 June 2024
Available online 17 June 2024
0950-7051/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

accuracy and reliability of the GNN model. Therefore, incorrect labels hinder the success of the model extraction attacks.

Second, commercial entities often limit the number of queries each user can make over a certain period of time. This is so that any would-be attackers can only gather labels for a small subset of nodes. Models trained with limited labeled nodes often show increased prediction errors, complicating the training of an accurate surrogate model. This issue is especially pronounced in GNN training, where sparse datasets exacerbate the problem. Unlabeled nodes primarily receive information from their topological neighbors. Due to graph sparsity, many nodes are unconnected, and some labeled node information is not transmitted to the required unlabeled nodes. Thus, executing successful model extraction attacks against GNNs with limited labeled nodes is highly challenging.

Finally, machine learning as a service (MLaaS) often operates in a black-box environment, where users have limited knowledge about the model's structure or training data. Black-box attacks targeting privacy are typically more difficult than white-box or gray-box attacks [20]. Consequently, black-box GNN model extraction attacks pose significant challenges. These difficulties have led to the belief that machine learning models for graph data are immune to model extraction attacks.

However, in this paper, we demonstrate that GNN models are still vulnerable against model extraction attacks, even in a black-box setting. We introduce an effective methodology that involves GNN model extraction in a black-box setting. The approach considers error responses and works within a limited number of queries. In this scenario, the attacker can only access specific class labels for node predictions, which we term *predicted labels*, rather than any intermediate layers or posterior probabilities. And, although the predicted labels may contain erroneous labels, the proposed attack methodology still proves to be effective despite this challenge.

In GNNs, information propagation heavily depends on the underlying graph topology. To address the issue of incorrect labels affecting the surrogate model, we propose a novel solution that involves modifying the graph's topology. More specifically, the method adds potential edges that connect labeled and unlabeled nodes. These edges are linked through node similarity measurements and a prediction model using an approach that connects correctly labeled nodes with unlabeled ones. This reduces the impact of incorrect labels during training.

Additionally, connecting labeled nodes with unlabeled nodes fully leverages the graph's topological information to train a superior surrogate model using only a limited number of queries. By adding potential edges, the surrogate model becomes more consistent with the target model. These potential edges facilitate information propagation, enhancing extraction performance with fewer queries and by better using the GNN's inherent information propagation capabilities.

In summary, this research makes the following contributions:

- We have designed a more realistic black-box environment for GNN model extraction attacks, which considers that the attacker will likely have limited access to the target model.
- We propose a GNN model extraction attack method that can be successful with a limited number of queries, even when the model's responses contain incorrectly predicted labels.
- Through a theoretical analysis and an empirical evaluation, we demonstrate the vulnerability of current GNN models.

2. Related work

2.1. Privacy attacks on machine learning

Model Extraction Attacks. In a model extraction attack, the adversary aims to create a surrogate model that replicates the behavior of the target model [14]. This can pose a significant privacy threat, especially when the target model is used in security-critical applications where sensitive data could potentially be exposed.

Numerous studies in the field of machine learning security highlight the danger of model extraction attacks. For example, Yan et al. [21] conducted a comprehensive analysis of such attacks with a focus on the factors that influence membership inference attacks. They examined four key contributors to a successful attack: the target model's task accuracy, the impact of the model architecture, its robustness, and the surrogate model's architecture. Additionally, they explored explainable model extraction attacks, providing a nuanced understanding of the attack mechanisms involved [22,23].

In response to these identified threats, researchers have developed defense strategies against model extraction attacks. For example, Chen et al. [24] introduced a pioneering defense mechanism with two key modules: interrupt detection and interrupt recovery. These modules aim to fortify the model against extraction attempts. Further, Li et al. [25] proposed a unique defense approach using physical unclonable functions (PUFs). Their scheme deploys PUF models at the service provider's side to ensure that only legitimate users can access accurate model predictions. As such, they have established a robust protection against model extraction attacks.

Membership Inference Attacks. Membership inference attacks aim to determine whether a specific data point was used to train a machine learning model [15]. These attacks exploit subtle information leaks to detect whether individual samples are included in the training set.

Several researchers have explored membership inference attacks, uncovering vulnerabilities in the machine learning models. Liu et al. [26], for instance, have made some significant contributions to this research stream, especially when it comes to black-box membership inference attacks. As an example, they introduced gradient leaks, showing that these attacks can be executed with black-box access alone. Typically, their research focuses on detecting whether specific data records are part of a target model's training set. In federated learning, Liu et al. [27] expanded their study, introducing a subject-level attack using data augmentation and model differences. Wang et al. [28] examined membership inference attacks in GNNs, exploring both their vulnerabilities and their defense mechanisms. They developed a white-box defense algorithm that includes a graph perturbation injection.

Attribute Inference Attacks. Attribute inference attacks involve deducing the sensitive attributes of individuals from a model's predictions [29]. Even if the model is not explicitly trained on these attributes, attackers can exploit patterns in the predictions to infer sensitive information. This poses significant privacy risks, as it allows attackers to uncover personal details that the model indirectly reveals.

The realm of attribute inference attacks has garnered considerable research attention. For example, Torra et al. [16] explore privacy leaks within k-Anonymity, focusing on numerical data. Using established rules to identify sensitive cells in tabular data, their study reveals that attribute leaks can extend beyond categorical data. Hu et al. [30] propose a federated evolutionary feature selection algorithm based on particle swarm optimization to address multi-participant feature selection under privacy protection. Researchers have also studied attribute inference attacks in the context of GNNs. Zhang et al. [31] introduce an attribute inversion attack method for scenarios where the adversary has a trained GNN model and auxiliary knowledge. Zhang et al. [32] further investigate attribute inference and graph reconstruction attacks using graph embeddings, presenting a novel algorithm based on graph neural networks to extract sensitive attributes from graph data.

2.2. Model extraction attacks on GNNs

Within the realm of privacy attacks, there is growing concern regarding the vulnerabilities of GNNs. Due to their prevalence in modeling graph-structured data, GNNs have become targets for various privacy threats. Model extraction attacks on GNNs have gained attention due to their potential to compromise the confidentiality of graph-based models.

Table 1
Comparison with previous GNN model extraction attacks. *No True Labels* indicates that the attacker does not have the correct labels for the existing data. *Predicted Labels* indicates that the target model response is only the predicted label value. *Limit Queries* indicates a restriction on the attacker's queries. *Incorrect Information* indicates consideration of error information in the response.

	No true labels	Predicted labels	Limit queries	Incorrect information
[17]	×	×	✓	×
[19]	✓	×	×	×
[18]	✓	×	✓	×
Our	✓	✓	✓	✓

Adversaries aim to construct surrogate GNN models by querying the target GNN, revealing sensitive information embedded in the model's architecture and representations. To this end, DeFazio et al. [17] formalized GNN extraction and proposed an initial solution. In a different approach, Shen et al. [19] examined model extraction attacks using distinct APIs. Wu et al. [18] explored these attacks under various knowledge scenarios, including node attributes, links, and auxiliary sub-graphs. They provided guidelines for executing attacks based on the available knowledge. These studies both enhance our understanding and bolster our defenses against model extraction attacks on GNNs.

2.3. Discussion

Table 1 showcases the advancements in GNN model extraction attacks. However, several areas need further improvement. For instance, David et al. [17] were successfully able to extract a well-performing model but, to do so, they required access to true data labels. Shen et al.'s [19] extraction effort required an unrealistic number of queries, while Wu et al. [18] ignored model extraction with predicted label responses. Hence, these studies do not adequately address the constraints of a black-box scenario where only the predicted labels are given as responses and incorrect predictions are included in the response data.

The model extraction attack presented in this paper introduces a novel approach that does not rely on the presence of true labels. Our methodology successfully extracts a model using only a limited number of queries. Plus, it leverages the responses from the target model, even when provided as predicted labels. It also accounts for the potential inclusion of incorrect labels.

3. Preliminaries

3.1. Notations

A graph is defined as $G = \{A, X\}$, where A denotes the adjacency matrix and X denotes the node feature matrix. Z denotes the hidden state of the feature vector, and z_v denotes the hidden state of node v. The hidden state is the node feature of the model network layer. A' denotes the predicted edge adjacency matrix, and $\hat{A}$ represents the adjacency matrix after adding potential edges. $\hat{Y}$ and Y_P denote the predicted probabilities and predicted labels, respectively, while T_s denotes the threshold of the cosine similarity, and T_a denotes the threshold that determines whether to create a potential edge. Here, u and v represent a single node of the graph. V_U and V_L are the sets of unlabeled and labeled nodes. $\mathcal{N}(v)$ represents the neighboring nodes of node v. Further, $S(X_v, X_u)$ denotes the cosine similarity of the nodes u and v. The notations used in this study are summarized in Table 2.

3.2. Model extraction attack

In machine learning privacy, adversaries aim to acquire information about either deployed models or training datasets. However, in model extraction attacks, the attacker's primary goal is to create a surrogate model that closely emulates the target model. This is achieved by iteratively querying the target model and using the obtained results to reverse-engineer its parameters and structure.

Table 2
Summary of notations.

Notations	Description
$G(X, A)$	Graph data
X	Node feature matrix
A	Adjacency matrix
Z	The hidden state of a feature vector
A'	The predicted edge adjacency matrix
$\hat{A}$	The adjacency matrix after adding edges
$\hat{Y}$	The model prediction probabilities
Y_P	Predicted labels
V	The set of nodes in the graph
V_U, V_L	The set of unlabeled and labeled nodes.
T_s, T_a	Threshold value
u, v	A single node in the graph
z_v	The hidden state of node v
$\mathcal{N}(v)$	The neighboring nodes of node v
X_v, X_u	The node features of v and u
$S(X_v, X_u)$	The cosine similarity of the u and v nodes

Here, several specialized terms need to be introduced and defined:

- **Target Dataset.** The target dataset refers to the data the service provider has used to train the target model.
- **Target Model (Victim Model):** The target model, also known as the victim model, is the focus of the model extraction attack. This model is trained by the service provider using a specific dataset and then deployed to users for them to interact with.
- **Predicted Label.** The target model responds to user queries in various formats, one of which is the predicted label. A predicted label represents the specific classification provided by the target model for the given query data. However, due to inherent imperfections in the target model, it is important to note that these predicted labels may not always be accurate.
- **Surrogate Dataset.** A surrogate dataset is a dataset belonging to the attacker that is used to train a surrogate model. The effectiveness of a model extraction attack is often increased when the distribution of the surrogate dataset closely mirrors that of the target dataset.
- **Surrogate Model.** The surrogate model, obtained through a model extraction attack, typically shares similar functionality and performance to the target model. Ideally, its predictions will closely match those of the target model.

The fundamental process of a model extraction attack is depicted in Fig. 1. Initially, the model owner trains the target model with the target dataset and deploys it to an MLaaS server. The model owner grants access to the target model interface to fulfill user requests. The attack begins when the local attacker forwards data (x) from the surrogate query dataset to the target model. The target model generates a prediction ($f(x)$) based on the query data and communicates the result to the attacker. The attacker then uses the data and responses to train the surrogate model. The primary objective is to replicate the target model's behavior, ensuring that both models provide consistent predictions, i.e., $f(x) = f'(x)$.

3.3. Graph neural networks (GNNs)

Given the crucial role of a graph's topology and the features of its nodes in representing graph-structured data, the task of representation learning for graphs is challenging. To address this, GNNs [7] have been developed to process graph-structured data as an extension of traditional neural network techniques. GNNs can effectively capture and aggregate information about graph topology and node properties, demonstrating impressive performance with various tasks at the node-, link-, and graph-level.

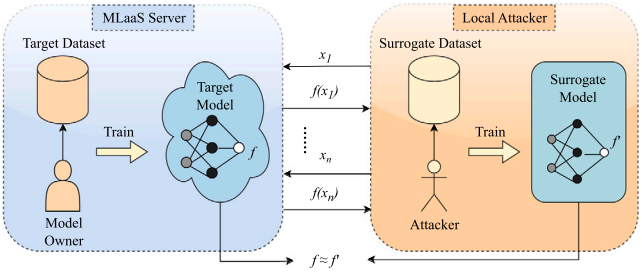


Fig. 1. Model extraction attack's fundamental process.

Through a message-passing mechanism, GNNs capture the topological information and node features of graphs. Specifically, the GNN aggregates data from neighboring nodes to update the node representation at each layer. The updated node representation in the kth layer of the GNN can be formulated as:

$$\begin{aligned} z_v^{(k)} &= \mathrm{COMBINE}^{(k)}\left(z_v^{(k-1)}, m_v^{(k)}\right) \\ m_v^{(k)} &= \mathrm{AGG}^{(k-1)}\left(\left\{z_u^{(k-1)} : \forall u \in \mathcal{N}(v) \cup v\right\}\right) \end{aligned} \tag{1}$$

where $z_v^{(k)}$ is the hidden state of node $v \in \mathcal{N}(v)$ in the kth layer, $z_v^{(0)}$ denotes the input feature of node v, $\mathcal{N}(v)$ denotes the set of neighboring nodes of node v, and u is an edge adjacent to these nodes. The function AGG aggregates the hidden states of neighboring nodes.

4. GNN model extraction problem definition

4.1. Extraction goal

The model owner trains a target model f with graph data $\mathrm{G(X, A)}$ and provides an API for users. The local attacker queries the target model by sending existing unlabeled data to the model. The model owner limits the number of queries and restricts the response information. Using the responses, the attacker trains a surrogate model f'. The trained surrogate model simulates the target model's functions, ensuring that $\forall v_i \in \mathrm{V}, f'\left(v_i\right) \approx f\left(v_i\right)$, where V is the set of nodes in the graph, and v_i is one of the nodes in the set.

In a model extraction attack, an attacker manages to create a surrogate model that operates similarly to the target model. Extraction can be considered successful under several circumstances — for instance, when the surrogate model achieves identical accuracy on the testing set ($f'\left(v_i\right) = y_i$) or when the model provides output predictions that are similar to those of the target model ($f'\left(v_i\right) \approx f\left(v_i\right)$). It is important to note that the structure and parameters of the surrogate model may not match those of the target model, even if they operate similarly. In other words, the weights or structure of the surrogate model may differ from those of the target model.

4.2. Extraction background knowledge

Many model owners enforce restrictions within their APIs to protect their interests. Therefore, to increase the realism of the attack scenario, we simulated a much tougher attack environment — one that more closely mimics the real world.

First, we imposed restrictions on the attacker's access by ensuring the attack had to be conducted within a black-box environment. In this black-box setting, the attacker cannot access predictions, model parameters, or labels. This scenario, with only minimal knowledge accessible by the attacker, forms a much closer comparison to existing attack scenarios.

Second, in the black-box setting, only the labels predicted by the target model can be accessed. As shown in Fig. 2, when submitting a query to the target model, only the predicted label of the corresponding node is retrieved. This limited response serves as an effective method of preserving privacy since these predicted labels only disclose a very limited amount of information. Additionally, these predicted labels may occasionally be incorrect, posing additional challenges to the attacker.

Finally, the service provider may enforce restrictions on the number of interface queries for commercial or privacy reasons. Therefore, to enhance the realism of the attack, we imposed a limit on the maximum number of queries an attacker could execute. This query allowance curtails the attacker's capacity to assign predicted labels to a subset of nodes. It is widely recognized that machine learning models typically demonstrate improved performance with larger datasets. Consequently, the constrained number of queries not only adds complexity to the model extraction procedure but also reduces the availability of predicted labels, thereby increasing the challenge of the task.

In conclusion, conducting model extraction attacks under the constraints of a black-box environment, limited query information, and restricted number of queries presents significant challenges. However, our novel extraction methodology overcomes these challenges, successfully extracting a well-performing GNN model despite the obstacles put in its way.

5. Extraction methodology

5.1. Overview of extraction methodology

An overview of our proposed extraction methodology is illustrated in Fig. 2. Our extraction method consists of two main modules: an edge prediction module and a surrogate model construction module.

The attacker employs two techniques in the edge prediction module to identify potential edges. The first technique assesses node similarity to determine if an edge exists between two nodes. The second technique involves training a new GNN model using available data and predicted label responses to predict the values of potential edges.

The identified potential edges are processed in the surrogate model construction module to create a new adjacency matrix. The attacker then trains the surrogate model using both the predicted labels and the updated adjacency matrix. The resulting surrogate model functions equivalently to the target model, producing predictions that match those of the target model.

In our proposed extraction methodology, the edge prediction module allows the surrogate model to effectively use the graph's topological information, enhancing information propagation by including potential edges. This approach helps mitigate the impact of erroneous predicted labels from API responses by using the graph's topology, which in turn leads to better model performance. Moreover, improved information propagation enables the model extraction attack to be completed with fewer nodes. The following section provides a comprehensive description of the model extraction technique.

5.2. The edge prediction

5.2.1. Candidate sets of potential edges

The first step in identifying plausible connections is to generate an initial set of candidate edges. This process involves assessing the suitability of establishing an edge between two nodes by computing the similarity of their node features. Dai et al. [33] highlight that high-performance models can be achieved by connecting unlabeled and labeled nodes with similar features. As such, we turned to cosine similarity, which is a key metric in GNN research and a very effective metric for assessing node similarity [34–36]. Thus, the candidate set is assembled by calculating the cosine similarity between unlabeled and labeled nodes.

More specifically, the cosine similarity between unlabeled and labeled nodes is calculated using their respective node features. This is formalized as:

$$\mathrm{S}(\mathrm{X}_v, \mathrm{X}_u) = \frac{\mathrm{X}_v \cdot \mathrm{X}_u}{\|\mathrm{X}_v\|_2 \cdot \|\mathrm{X}_u\|_2}, \forall u \in \mathcal{N}(v) \backslash v \tag{2}$$

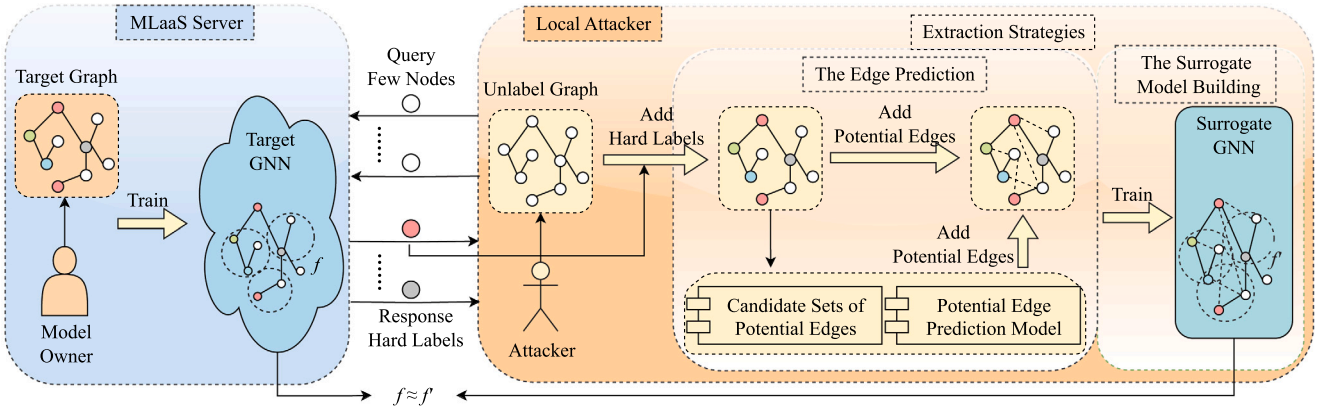


Fig. 2. Overview of the model extraction attack. Unlabeled nodes are indicated by hollow circles. Labeled nodes are indicated by solid circles. The solid line connecting the nodes indicates that edges originally existed between nodes. The dashed line connecting the nodes indicates potential edges predicted by the edge prediction model and post-processing.

where v denotes a labeled node, and u is an unlabeled node with no link to v. X_v and X_u represent the node features of v and u, respectively.

After calculating the cosine similarity, the resulting values are compared with a predetermined threshold T_s. If the cosine similarity score between two nodes exceeds this threshold, the nodes are considered similar, and an edge between them is added to the set of candidate edges.

In a model extraction attack, connecting unlabeled nodes with similarly labeled nodes allows us to leverage accurate labels, mitigating the adverse effects of inaccuracies in the predicted labels. Furthermore, GNNs extract features from graph data by aggregating information from nodes and their neighbors. Hence, increasing connectivity enhances the information transfer capability of GNNs, which means the model extraction attack can be completed with fewer queries. It is crucial to note that connecting dissimilar nodes does not provide any advantage, whereas thoughtfully selecting edges from the candidate set can mitigate the negative impact of dissimilar nodes on the model.

However, potential edges should be assigned less significance than the original edges of the graph. Directly giving potential edges the same weight as the original edges can reduce the latter's influence. Therefore, weights are assigned to the potential edges as a function of their value, as explained in the next section.

5.2.2. The values of potential edges

An additional GNN predicts the value of each potential edge by considering both the features of the connected node and the graph's topology. First, the GNN model extracts any embedding features that incorporate node features or topological information so as to improve edge prediction accuracy. This method, which is more effective than relying only on the original node features, is governed by the following formula:

$$\mathrm{A}' = \sigma\left(\mathrm{Z}^{\mathrm{T}}\mathrm{Z}\right), \mathrm{Z} = GNN(\mathrm{A}, \mathrm{X}) \tag{3}$$

where Z is the node representation feature vector, σ is the activation function, and A' is the predicted potential edge adjacency matrix.

In the original binary adjacency matrix of the graph, a value of 1 represents an edge, while 0 indicates no edge. Due to the matrix's sparsity, not all nodes are interconnected, while some have very few connections. This results in more zeros than ones, which can dominate the loss function and cause the edge predictor to default to 0, reducing prediction accuracy. To address this, negative sampling [37] is used during training. For positive samples where $\mathrm{A}'_{ij} = 1$, K nodes not connected to node v_j are randomly selected as negative samples. The loss function is defined as follows:

$$\mathcal{L}_E = \sum_{v_i \in \mathrm{V}} \sum_{v_j \in \mathcal{N}(v_i)} \left(\left(\mathrm{A}'_{ij} - 1\right)^2 + \sum_{n=1}^{K} \mathbb{E}_{v_n \sim P_n(v_i)} \left(\mathrm{A}'_{in} - 0\right)^2 \right) \tag{4}$$

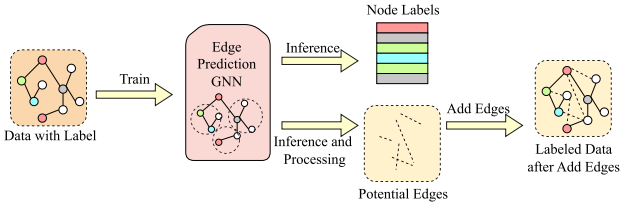


Fig. 3. Framework of the edge prediction model.

where $\mathcal{N}(v_i)$ represents the neighbors of node v_i and $P_n(v_i)$ is the distribution of nodes that have no link with v_i.

With the GNN-based edge predictor trained using Eq. (4), the values of the potential edges that link unlabeled nodes to labeled nodes can now be predicted. This mechanism enhances the robustness of the classification process even with limited and inaccurate labels.

Multi-task Learning for GNN Model. To make the node representations in the GNN-based edge predictor more meaningful, a node classification task f_C is conducted alongside the edge prediction task. As depicted in Fig. 3, the potential edge values obtained through multi-task learning, including node classification, are better adapted to this task. Incorporating a node classification task into the process improves final accuracy and contributes to better model extraction results. The loss function for the node classification is expressed as:

$$\mathcal{L}_C = \sum_{y_i \in \mathrm{Y_P}} Loss\left(y'_i, y_i\right), \mathrm{Y}' = Softmax(GNN(\mathrm{A}, \mathrm{X})) \tag{5}$$

A prediction model assigns values to potential edges in the candidate set. Following this assignment, these potential edges help to train the GNN model, enhancing its performance but without diminishing the impact of the original edges.

5.2.3. Addition of potential edges

The resulting candidate set of potential edges and their values is then post-processed to derive a new adjacency matrix $\hat{\mathrm{A}}$. Subsequently, both the adjacency matrix $\hat{\mathrm{A}}$ and the original node features X are input into the GNN to train the surrogate model. The process is detailed below.

Initially, all the original edges in $\hat{\mathrm{A}}$ are included in the input by setting $\hat{\mathrm{A}}_{ij} = 1$ if $\mathrm{A}_{ij} = 1$. A double thresholding method is used to determine whether to add a potential edge between any unlabeled nodes $v_i \in \mathrm{V_U}$ and the labeled nodes $v_j \in \mathrm{V_L}$ that were not originally connected. The first threshold checks whether the cosine similarity of two nodes is greater than the threshold T_s, i.e., whether $S(X_{v_i}, X_{x_j}) > T_s$. The second threshold checks whether the value of the potential edge obtained from the prediction is greater than the threshold T_a,

i.e., whether $A'_{ij} > T_a$. If the result exceeds both thresholds, the original value of 0 for the two nodes in the adjacency matrix is replaced with the value of the potential edge obtained from the prediction, i.e., $\hat{A}_{ij} = A'_{ij}$. However, if the result does not exceed one or more of the thresholds, $\hat{A}_{ij}$ is set to 0 in the original neighbor matrix. This post-processing step helps to mitigate the negative impact of irrelevant edges on the surrogate model's training. The process is formulated as follows:

$$\hat{A}_{ij} = \begin{cases} 1 & \text{if } v_j \in \mathcal{N}(v_i); \\ A'_{ij} & \text{else if } S(X_{v_i}, X_{x_j}) > T_s, A'_{ij} > T_a, \\ & \text{and } v_i \in V_U, v_j \in V_L; \\ 0 & \text{else} \end{cases} \tag{6}$$

where A' is the value of the potential edge obtained from the prediction model, and $\hat{A}_{ij}$ is the adjacency matrix after adding potential edges. A is replaced with $\hat{A}$ to train the surrogate model.

5.3. The surrogate model building

The main objective of the model extraction attack is to obtain a surrogate model with similar functionality and performance to the target model. In our extraction methodology, adding edges enhances the surrogate model's performance. Hence, the post-processed adjacency matrix $\hat{A}$, the node features X, and the predicted labels Y_P are all input into the model. Then the surrogate model is trained. The training process can be formally expressed as:

$$L_A = \sum_{y_i \in Y_P} Loss\left(\hat{y}_i, y_i\right), \hat{Y} = Softmax(GNN(\hat{A}, X)) \tag{7}$$

where $\hat{Y}$ is the final predicted label. With the edge prediction model adding potential edges to facilitate the information propagation and the extraction attack model predicting the labels, the overall loss function can be written as:

$$\mathcal{L}_{total} = \alpha \mathcal{L}_A + \beta \mathcal{L}_E + \gamma \mathcal{L}_C \tag{8}$$

where α, β and γ are hyperparameters, and depending on the loss value and the function.

Algorithm 1 outlines the training procedure for our proposed extraction strategies. In line 3, the cosine similarity between the nodes is calculated first using Eq. (2). Then, lines 5–8 describe the process of optimizing the neural network. Next, the edge prediction model f_E and the node classification layer f_C are generated using Eqs. (4) and (5), respectively. The potential edges are then predicted using f_E, which produces the potential edge adjacency matrix A'. A' is then filtered to obtain $\hat{A}$ using Eq. (6), after which the surrogate model f_A is trained using Eq. (7). Lastly, f_E, f_C, and f_A are all jointly trained using an Adam optimizer [38] with a learning rate of 0.001.

Our proposed methodology advances traditional surrogate model training by incorporating potential edges and leveraging graph topology in GNNs for message propagation. This approach effectively addresses the challenge of incorrectly predicted labels by connecting unlabeled nodes with correct labels. Additionally, the increased propagation of the connected information produces a high-performing surrogate model, even given limited query access.

5.4. Theoretical proof

Topological features are crucial in graph data as they effectively express the links between nodes. To reduce the impact of incorrect labels, some unlabeled nodes are linked to labeled nodes. A GNN model predicts where these edges will occur. By connecting labeled and unlabeled nodes, the GNN can fully leverage the knowledge from otherwise unlabeled nodes to improve prediction accuracy.

Given a multilayer GNN model with node classifications, node representations in the kth layer, i.e., $Z = H^{(k)}W^{(K)}$, can be considered as the nodes' predictions, with the final prediction produced by aggregating Z.

Algorithm 1 Extraction Strategies.

1: **Input:** G(X, A), Y_P, T_s, T_a, α, β, and γ
2: **Output:** f_E, f_C, and f_A
3: Compute the cosine similarity S by Eq. (2)
4: **repeat**
5: Obtain A' with Eq. (4) and Eq. (5)
6: Obtain $\hat{A}_{ij}$ with Eq. (6)
7: Obtain the surrogate model f_A with Eq. (7)
8: Jointly optimize the parameters of f_E, f_C, and f_A by Eq. (8)
9: **until** convergence
10: **return** f_E, f_C, and f_A

In a GNN, the predicted probability of an unlabeled node u belongs to the class c can be expressed formally as:

$$\hat{y}_{uc} = \frac{1}{m+n}\left(\sum_{a \in V_a} z_{ac} + \sum_{l \in V_n} z_{lc}\right) \tag{9}$$

where V_a denotes the set of unlabeled neighbors of u, and m denotes the number of these neighbors. Similarly, V_n denotes the set of labeled neighbors of u, and n denotes the number of these neighbors. Z can be treated as the predictions of the nodes, where z_{ac} and z_{lc} denote the predicted probability of belonging to class c for $a \in V_a$ and $l \in V_n$, respectively. Moreover, the GNN's predictions are produced by aggregating Z.

5.4.1. Proof that predicted label responses yield more robust models

Adding edges improves the model's accuracy and reduces the effect of incorrect labels when training the surrogate model. In the case of predicted labels, the average value of $\hat{y}_{uc}$ can be expressed as:

$$\begin{aligned} \mathbb{E}\left(\hat{y}_{uc}\right) = & \frac{m}{m+n}\mathbb{E}\left(z_{ac}\right) + \frac{(hp_t + (1-h)p_f)m}{m+n}\mathbb{E}\left(z_{bc}\right) \\ & + \frac{(h(1-p_t) + (1-h)(1-p_f))m}{m+n}\mathbb{E}\left(z_{dc}\right) \end{aligned} \tag{10}$$

where z_{ac} denotes the contribution of the unlabeled node $a \in V_a$ to node u for class c. Similarly, z_{bc} denotes the contribution of labeled nodes $b \in V_n$ with label c and z_{dc} denotes the contribution of labeled nodes $d \in V_n$ with labels not equal to c. The parameter h represents the ratio of intra-class edges. Note that the symbols used in the subsequent proof have the same meaning as those used here.

In addition, let p_t denote the probability that the predicted label c is correct, i.e., that a node belonging to the class c has been assigned a predicted label c. Conversely, p_f denotes the probability that the predicted label c is incorrect, i.e., that a node not belonging to the class c has been assigned this label. As $p_t > p_f$, we can have $p = (hp_t + (1-h)p_f) < p_t$. Therefore, Eq. (10) can be rewritten as:

$$\mathbb{E}\left(\hat{y}_{uc}\right) = \frac{n\mathbb{E}\left(z_{ac}\right) + pm\mathbb{E}\left(z_{bc}\right) + (1-p)m\mathbb{E}\left(z_{dc}\right)}{m+n} \tag{11}$$

We further connect node u to k similarly labeled nodes. Then we can obtain the corresponding predicted probability $\hat{y}^k_{uc}$ as:

$$\mathbb{E}\left(\hat{y}^k_{uc}\right) = \frac{m+n}{m+n+k}\mathbb{E}\left(\hat{y}_{uc}\right) + \frac{kp_t\mathbb{E}\left(z_{bc}\right) + k\left(1-p_t\right)\mathbb{E}\left(z_{dc}\right)}{m+n+k} \tag{12}$$

As the proof of [33], we could have $\mathbb{E}\left(\hat{y}^k_{uc}\right) > \mathbb{E}\left(\hat{y}_{uc}\right)$ when accuracy of predicted labels $p_t > \frac{\mathbb{E}(s_{ac}) - \mathbb{E}(s_{dc})}{\mathbb{E}(s_{bc}) - \mathbb{E}(s_{dc})}$. Therefore, we can conclude that linking the unlabeled nodes with the predicted labeled nodes can reduce the impact of incorrect labels and improve the model's accuracy.

5.4.2. Proof that a limited number of responses yields a higher accuracy model

In the case of clean labels, the average value of $\hat{y}_{uc}$ would be:

$$\mathbb{E}\left(\hat{y}_{uc}\right) = \frac{m}{m+n}\mathbb{E}\left(z_{ac}\right) + \frac{hn}{m+n}\mathbb{E}\left(z_{bc}\right) + \frac{(1-h)n}{m+n}\mathbb{E}\left(z_{dc}\right) \tag{13}$$

When linking node v_u with k similarly labeled nodes, the corresponding predicted probability $\hat{y}_{uc}^k$ can be determined through:

$$\mathbb{E}\left(\hat{y}_{uc}^k\right) = \frac{m+n}{m+n+k}\mathbb{E}\left(\hat{y}_{uc}\right) + \frac{hk\mathbb{E}\left(z_{bc}\right) + (1-h)k\mathbb{E}\left(z_{dc}\right)}{m+n+k} \tag{14}$$

where k is the number of edges added. Then, combining Eq. (13) with Eq. (14), we can derive:

$$\mathbb{E}\left(\hat{y}_{uc}^k\right) - \mathbb{E}\left(\hat{y}_{uc}\right) = -\frac{km\mathbb{E}\left(z_{ac}\right)}{(m+n+k)(m+n)} + \frac{hkm\mathbb{E}\left(z_{bc}\right) + (1-h)km\mathbb{E}\left(z_{dc}\right)}{(m+n+k)(m+n)} \tag{15}$$

When $h\mathbb{E}\left(z_{bc}\right) + (1-h)\mathbb{E}\left(z_{dc}\right) > \mathbb{E}\left(z_{ac}\right)$, we could have $\mathbb{E}\left(\hat{y}_{uc}^k\right) > \mathbb{E}\left(\hat{y}_{uc}\right)$. Therefore, we can conclude that linking unlabeled nodes to similarly labeled nodes will improve the model's performance. As shown in Eq. (15), the performance improvement is explicitly related to the value of k, which is the number of edges added.

Comparing Eqs. (13) and (14), the predicted probability increases with as k increases. With a small number of labeled nodes n, adding links can increase information acquisition and improve model accuracy. Therefore, we can successfully complete the model extraction attack and procure an excellent surrogate model in only a few queries by adding edges during the extraction model's training.

5.4.3. Proof of restricted black-box efficiency

In this paper's model extraction attack, the attacker starts with unlabeled data and then queries the acquired predicted labels to train the surrogate model. The training of the surrogate model can be represented as follows:

$$\mathcal{L} = \sum_{y_i \in \mathrm{Y_P}} Loss\left(\hat{y}_i, y_i\right), \hat{\mathrm{Y}} = Softmax(GNN(\hat{\mathrm{A}}, \mathrm{X})) \tag{16}$$

where the GNN is a generic structure that can be adapted to different GNN architectures, such as a graph convolutional network (GCN) [39], a simplified graph convolutional network (SGC) [40], an ARMA graph neural networks (ARMA) [41], or something else. In our experiments, we constructed a surrogate model using these GNN architectures as our base. This implies that we successfully executed a series of model extraction attacks without having prior knowledge of the target model's structure. In other words, our approach is effective in the case of restricted black-box scenarios.

In previous works such as [17,19], a cross-entropy loss function was used to calculate the training loss of the model. The equation below represents the cross-entropy loss function:

$$\mathcal{L}_{CE}\left(y_p, \hat{y}\right) = -\left[y_p \log \hat{y} + \left(1-y_p\right)\log\left(1-\hat{y}\right)\right] \tag{17}$$

where y_p is the target model's predicted label, which attackers use to train surrogate models. In other words, these studies require access to the output probabilities of the target model. However, in this work, the only necessary output is the class, which is a more realistic requirement.

5.5. Case study

Fig. 4 presents a case study on the proposed model extraction attack. The shaded area in the figure illustrates the process of updating the attacker's data. As such, it highlights the progression of the attack.

In the case study, the attacker possesses a graph with 10 nodes but does not know their labels. The attacker then uses some unlabeled data to query the target model. It is important to note that the target model, according to this paper's constraints, provides only a limited number of predicted labels to the user. In this case study, that number is five, i.e., the target model assigns predicted labels to five nodes, transmitting them to the user. However, due to the inherent imperfections of the target model, one of the labels provided for the five nodes is incorrect. The attacker adjusts the existing graph data based on the acquired predicted labels, creating an updated graph where certain nodes now have correct labels.

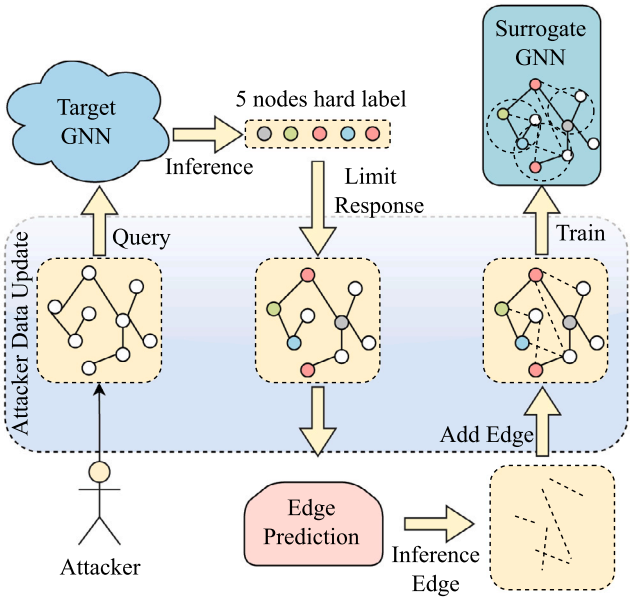


Fig. 4. Case study of the model extraction attack method.

Table 3
Datasets and attack taxonomy.

	Train/Val	Nodes	Edges	Attack-0	Attack-1	Attack-2	Attack-3
Cora	140/300	2708	10,556	35	70	105	140
Citeseer	120/500	3327	9228	30	60	90	120
Pubmed	60/500	19,717	88,651	15	30	45	60
Physics	–	34,493	495,924	86	172	258	344
Ogbn-arxiv	–	169,343	1,116,243	423	846	1269	1693

Next, the attacker uses the labeled graph to initiate a model extraction attack. The graph data is first sent to the edge prediction module, which predicts potential edges. The edge prediction module identifies five potential edges in the case study, as shown in the figure. The attacker then incorporates these potential edges into the existing labeled graph, modifying the graph data. After inserting the edges, the attacker trains a surrogate model on the modified labeled graph data. This training process aims to create a surrogate model that effectively mimics the functionality of the target model.

6. Experiments

6.1. Experimental setup

6.1.1. Datasets

We employed three publicly available datasets to assess the efficacy of our attack methodology, namely Cora, Citeseer, and Pubmed [39]. These datasets have been extensively used as benchmarks for evaluating node classification models. They consist of citation networks, where the nodes correspond to publications, and the edges represent citations between these publications. Additionally, we conducted experiments on larger graphs such as Physics [42] and Ogbn-arxiv [43], which feature more nodes and edges. Table 3 provides an overview of the dataset statistics.

6.1.2. Dataset configuration

The datasets were each partitioned into training, validation, and testing sets. Each testing set was used across all models to assess performance. Likewise, the training sets were used to train the target models.

Table 4
Results of each method on different datasets with default training set sizes.

Dataset	GCN		SGC		ARMA		OUR	
	Accuracy	Fidelity	Accuracy	Fidelity	Accuracy	Fidelity	Accuracy	Fidelity
Cora	75.30 ± 1.36	79.60 ± 0.97	74.24 ± 1.30	78.48 ± 0.82	74.06 ± 2.22	77.62 ± 2.07	**80.76 ± 1.34**	**83.04 ± 2.22**
Citeseer	67.40 ± 0.99	73.72 ± 1.65	65.66 ± 1.24	71.50 ± 2.57	66.52 ± 0.61	72.28 ± 0.78	**69.74 ± 1.33**	**77.88 ± 1.82**
Pubmed	75.76 ± 0.13	80.48 ± 0.71	75.58 ± 0.19	80.62 ± 1.24	73.70 ± 0.40	77.72 ± 0.87	**78.20 ± 1.23**	**86.12 ± 1.38**

In the process of extracting the model, nodes from the validation set were randomly selected, and the target model was queried to acquire the predicted labels. Subsequently, the surrogate models were trained using the validation set, which incorporates the obtained predicted labels.

To make these model extraction attacks more realistic, we limited the number of queries each user could perform to mimic the tactics commonly used by providers to protect their commercial IP and privacy. Hence, distinct query limits were set for each dataset, resulting in varying query numbers for each attack. As illustrated in Table 3, we numbered each attack to delineate between the different numbers of queries allowed. i.e., Attack-0, Attack-1, Attack-2, and Attack-3. Here, we describe each attack:

- **Attack-0**. In this attack, we set the number of queries to 35, 30, and 15 for Cora, Citeseer, and Pubmed, respectively. These numbers represent one-fourth of the default number of training samples in each dataset. For the larger datasets Physics and Ogbn-arxiv, we set the number of queries to 86 and 423, respectively, equivalent to 1/400 of their nodes.
- **Attack-1**. In this attack, we set the number of queries to 70, 60, and 30, respectively. These numbers are equivalent to half the default number of training samples in each dataset. For the larger datasets, we set the number of queries to 172 and 846, respectively.
- **Attack-2**. In this attack, we set the number of queries to 105, 90, and 45, respectively, corresponding to three-fourths of the default number of training samples in each dataset. For the larger datasets, we set the number of queries to 258 and 1,269, respectively.
- **Attack-3**. Here, the attackers were allowed 140, 120, and 60 queries, respectively, reflecting each dataset's default number of training samples. For the larger datasets, we set the number of queries to 344 and 1,693, respectively, which is equivalent to 1/100 of their nodes.

6.1.3. Models

We employ the widely-used graph convolutional network (GCN) [39] as the foundational model for our extraction methodology. Furthermore, to assess the generality of our proposed method, we conduct experiments with an SGC [40] and an ARMA [41]. The hidden layer in all networks was set to 16, except for the edge prediction model, which featured 64 neurons in the hidden layer. The remaining configuration was consistent across all networks, with two layers in total. The activation function used for the hidden layer was ReLU, and the output layer employed softmax. Training was conducted with the Adam optimizer at a learning rate of 0.001 for 1600 epochs.

6.1.4. Evaluation metrics

Following the evaluation methodologies employed in prior model extraction attacks on deep neural networks (DNNs) [44,45], we evaluated performance using two metrics: accuracy and fidelity.

- **Accuracy.** Accuracy measures the level of correctness exhibited by the surrogate models on the testing data, defined as $f'(v_i) = y_i$. This metric is computed by dividing the number of nodes correctly classified by the total number of nodes in the testing set. A higher accuracy rate signifies better performance by the surrogate model.
- **Fidelity.** Fidelity assesses the similarity between the surrogate models and the target models on the testing data, quantified as $f'(v_i) \approx f(v_i)$. This metric is computed by dividing the number of predictions where both models produce the same result by the total number of test inputs. A higher fidelity indicates a stronger resemblance in performance between the surrogate model and the target model.

6.2. Effectiveness validation of proposed attack methods

To evaluate the effectiveness of our proposed GNN model extraction attack method, we compare it to the extraction attack method in the Attack-3 scenario using the original GCN, SGC, and ARMA models. The results of these methods on the Cora, Citeseer, and Pubmed datasets are presented in Table 4.

The table reveals that, in terms of accuracy and fidelity, the approach introduced in this paper outperforms the GCN, SGC, and ARMA methods. On the Cora dataset, our method yielded an accuracy and fidelity of over 80%, representing an average improvement of 5%. With Pubmed, our method enhanced fidelity by 6%, and in the case of Citeseer, both accuracy and fidelity saw a 3% improvement.

The experimental results further demonstrate that our proposed method enhances the efficacy of model extraction attacks. This improvement is achieved by establishing connections to more accurate labels and mitigating the adverse impact of incorrect labels through these connections.

6.3. Limiting the number of queries

This section examines how effective our proposed method is given limited queries on the target model. Here, we conducted experiments with all three datasets, varying the number of allowed queries as per Attacks-0-3. The results of these experiments are presented in Table 5.

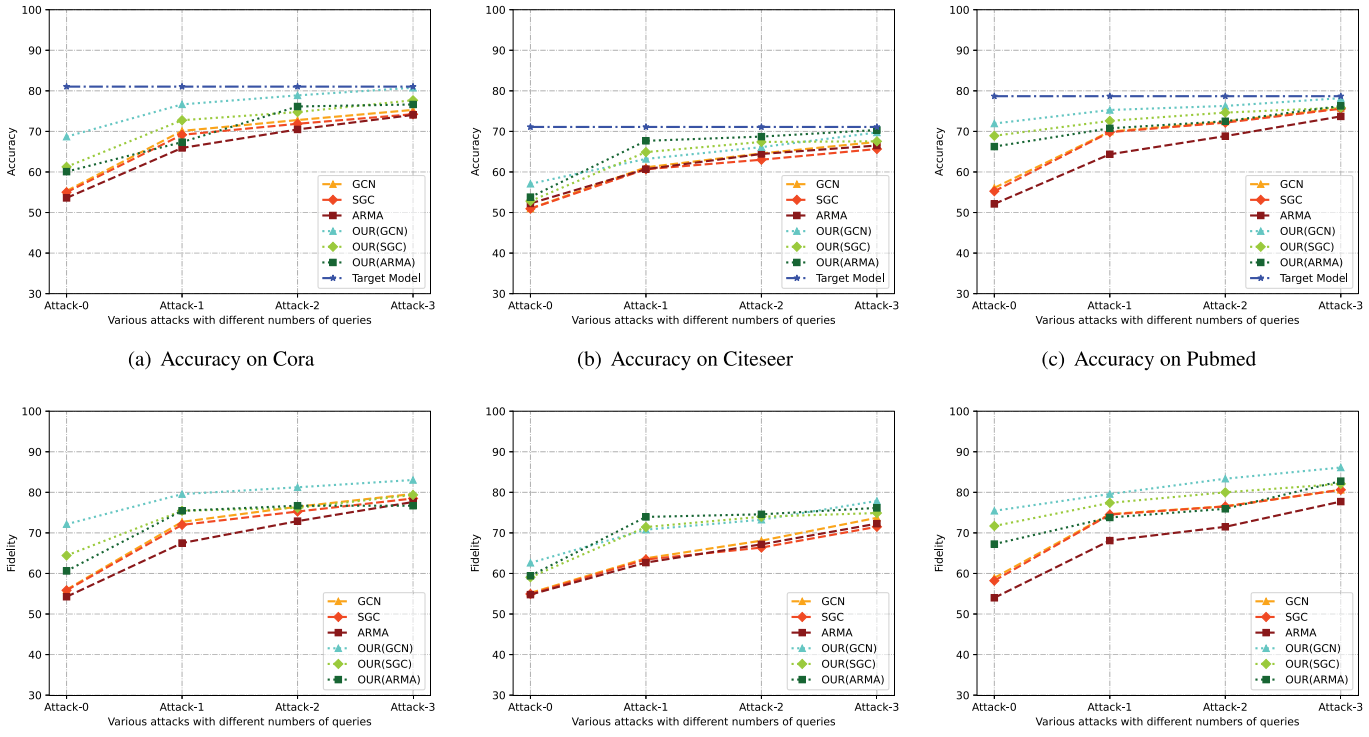
The table illustrates that our method consistently outperforms the original GCN-based attacks and other methods no matter the number of queries allow. This highlights the robustness and superiority of our approach. For example, in Attack-3, our method almost matched the accuracy of the target model. Additionally, on Pubmed, our method reached a fidelity of 86.12%, which is a very strong performance for a model extraction attack. Fig. 5 visualizes the superior performance of our method in each attack scenario. The figure also shows that the performance of each method improves as the number of queries increases, which reinforces the necessity for service providers to continue limiting the number of queries users are allowed to make for privacy protection purposes.

Notably, in Attack-0, our method shows significantly better performance than the comparators. With Cora, our method improved accuracy and fidelity by an average of 5%, while, on Pubmed, our method achieved a 10% improvement.

Moreover, our method was able to provide comparable results with fewer queries than the original method. For instance, in Attack-2, our method yielded performance comparable to the results other methods delivered for Attack-3. Additionally, with Cora in Attack-0, our method yielded an accuracy of 76.66%, outperforming the GCN's result of 72.80% in Attack-2 and 75.30% in Attack-3. Similar trends can be observed in several other cases. These findings underscore that our method can effectively execute model extraction attacks even with a restricted number of query nodes.

Table 5
Results of each method in Attack-0 to Attack-3 with GCN as the target model.

Metrics		Accuracy			Fidelity		
Dataset		Cora	Citeseer	Pubmed	Cora	Citeseer	Pubmed
Target Model		81.04 ± 0.34	71.10 ± 0.29	78.68 ± 0.87	–	–	–
Attack-0	GCN	55.32 ± 3.40	50.98 ± 2.76	56.08 ± 2.85	56.00 ± 2.66	55.20 ± 2.34	58.90 ± 2.37
	SGC	55.00 ± 3.82	50.94 ± 4.75	55.26 ± 2.59	55.84 ± 3.45	54.94 ± 5.00	58.22 ± 2.47
	ARMA	53.60 ± 1.66	52.24 ± 3.53	52.12 ± 2.10	54.26 ± 1.60	54.78 ± 3.09	54.00 ± 1.61
	OUR (GCN)	68.70 ± 3.01	57.08 ± 3.17	71.94 ± 2.82	72.14 ± 3.56	62.62 ± 3.32	75.44 ± 2.49
	OUR (SGC)	61.26 ± 3.99	52.82 ± 3.01	68.92 ± 3.42	64.42 ± 3.71	59.06 ± 3.79	71.70 ± 5.37
	OUR (ARMA)	60.08 ± 6.46	53.80 ± 1.33	66.26 ± 3.46	60.62 ± 6.00	59.42 ± 3.13	67.24 ± 2.38
Attack-1	GCN	70.12 ± 0.93	61.14 ± 0.66	69.96 ± 0.72	72.72 ± 1.19	63.74 ± 0.98	74.64 ± 0.56
	SGC	69.16 ± 1.59	60.68 ± 3.44	69.82 ± 1.04	71.98 ± 1.48	63.44 ± 3.62	74.52 ± 1.22
	ARMA	65.92 ± 5.37	60.70 ± 1.72	64.36 ± 0.98	67.50 ± 5.84	62.68 ± 1.52	68.10 ± 0.80
	OUR (GCN)	76.66 ± 1.03	63.22 ± 1.70	75.28 ± 2.59	79.56 ± 1.68	70.80 ± 3.45	79.58 ± 4.37
	OUR (SGC)	72.76 ± 4.82	64.90 ± 2.79	72.60 ± 1.15	75.46 ± 3.83	71.44 ± 3.23	77.40 ± 1.75
	OUR (ARMA)	67.36 ± 2.84	67.68 ± 3.69	70.76 ± 1.47	68.80 ± 3.93	73.94 ± 2.92	73.80 ± 1.41
Attack-2	GCN	72.80 ± 0.52	64.58 ± 2.63	72.42 ± 0.49	76.46 ± 1.14	68.08 ± 2.68	76.56 ± 0.71
	SGC	71.88 ± 0.85	63.02 ± 2.09	72.16 ± 0.35	75.26 ± 1.47	66.38 ± 2.74	76.52 ± 0.60
	ARMA	70.48 ± 1.82	64.44 ± 1.03	68.84 ± 1.11	72.90 ± 2.30	67.18 ± 2.26	71.50 ± 0.62
	OUR (GCN)	78.88 ± 2.13	66.12 ± 2.02	76.28 ± 1.11	81.24 ± 1.68	73.24 ± 2.26	83.36 ± 2.22
	OUR (SGC)	74.76 ± 2.70	67.40 ± 2.35	74.64 ± 2.03	76.16 ± 1.32	73.98 ± 2.73	80.00 ± 2.82
	OUR (ARMA)	76.14 ± 2.31	68.72 ± 3.32	72.50 ± 1.41	76.70 ± 2.79	74.58 ± 2.33	75.94 ± 3.71
Attack-3	GCN	75.30 ± 1.36	67.40 ± 0.99	75.76 ± 0.13	79.60 ± 0.97	73.72 ± 1.65	80.48 ± 0.71
	SGC	74.24 ± 1.30	65.66 ± 1.24	75.58 ± 0.19	78.48 ± 0.82	71.50 ± 2.57	80.62 ± 1.24
	ARMA	74.06 ± 2.22	66.52 ± 0.61	73.70 ± 0.40	77.62 ± 2.07	72.28 ± 0.78	77.72 ± 0.87
	OUR (GCN)	80.76 ± 1.34	69.74 ± 1.33	78.20 ± 1.23	83.04 ± 2.22	77.88 ± 1.82	86.12 ± 1.38
	OUR (SGC)	77.68 ± 3.04	67.56 ± 2.67	75.86 ± 1.67	79.36 ± 4.01	74.90 ± 3.94	82.08 ± 2.45
	OUR (ARMA)	76.64 ± 0.71	70.34 ± 1.42	76.28 ± 2.26	76.70 ± 1.10	76.16 ± 1.40	82.76 ± 4.05



(a) Accuracy on Cora (b) Accuracy on Citeseer (c) Accuracy on Pubmed

(d) Fidelity on Cora (e) Fidelity on Citeseer (f) Fidelity on Pubmed

Fig. 5. Performance of each attack method given different numbers of queries.

6.4. Analysis of attack effectiveness under different model architectures

To validate the generality of our proposed method, we not only employed a GCN as the foundational architecture for our surrogate model, we also built and tested surrogate models based on SGC and ARMA. The results are presented in Table 5. Furthermore, by relaxing certain assumptions, we conducted experiments on model extraction attacks using SGC and ARMA as the target models, with detailed results provided in Table 6.

6.4.1. Different surrogate model architectures

We validate the generality of our proposed method by employing GCN, SGC, and ARMA as surrogate model architectures. Table 5 presents the results for OUR (GCN), OUR (SGC), and OUR (ARMA).

Table 6
Results of each attack method under different target models.

Metrics		Accuracy			Fidelity		
Dataset		Cora	Citeseer	Pubmed	Cora	Citeseer	Pubmed
Target Model (SGC)		80.92 ± 0.41	70.98 ± 0.28	79.00 ± 0.45	–	–	–
Attack-0	GCN	55.66 ± 2.88	49.48 ± 5.26	56.28 ± 2.10	55.68 ± 2.66	54.26 ± 5.13	58.54 ± 2.08
	SGC	54.16 ± 1.25	49.52 ± 4.69	54.14 ± 1.47	54.68 ± 1.23	55.30 ± 4.49	56.72 ± 1.42
	ARMA	52.96 ± 3.13	48.58 ± 4.80	53.62 ± 0.40	53.02 ± 3.21	53.00 ± 4.52	55.06 ± 0.42
	OUR (GCN)	63.78 ± 4.23	56.04 ± 3.10	71.92 ± 1.62	64.96 ± 5.13	62.08 ± 3.39	75.82 ± 2.25
	OUR (SGC)	62.70 ± 2.52	51.72 ± 2.27	69.26 ± 2.31	67.12 ± 3.43	57.74 ± 2.14	72.06 ± 3.40
	OUR (ARMA)	65.86 ± 5.21	58.50 ± 6.10	67.78 ± 3.03	66.68 ± 6.29	65.02 ± 4.55	68.56 ± 3.22
Attack-1	GCN	69.64 ± 0.93	60.50 ± 1.33	71.14 ± 0.77	72.44 ± 1.64	64.60 ± 1.44	74.88 ± 0.61
	SGC	68.94 ± 0.87	59.16 ± 0.55	71.28 ± 0.62	71.72 ± 1.06	62.76 ± 1.38	74.68 ± 1.04
	ARMA	67.30 ± 1.15	60.40 ± 1.20	64.96 ± 2.96	69.00 ± 1.89	62.66 ± 2.15	67.90 ± 2.44
	OUR (GCN)	70.18 ± 2.21	64.72 ± 2.15	75.20 ± 1.11	72.40 ± 3.06	70.98 ± 3.22	79.62 ± 1.76
	OUR (SGC)	72.42 ± 5.22	61.40 ± 1.46	74.48 ± 0.29	76.46 ± 5.96	67.66 ± 1.17	79.88 ± 1.01
	OUR (ARMA)	68.15 ± 1.88	65.02 ± 4.61	70.96 ± 3.16	71.58 ± 3.40	69.04 ± 4.58	74.82 ± 4.54
Attack-2	GCN	73.08 ± 1.69	63.44 ± 3.12	72.84 ± 0.76	76.84 ± 1.75	67.96 ± 3.20	76.54 ± 0.61
	SGC	70.20 ± 1.76	62.96 ± 3.30	72.66 ± 0.86	73.54 ± 1.77	67.70 ± 2.98	76.24 ± 0.74
	ARMA	70.44 ± 2.84	63.56 ± 2.57	68.96 ± 0.83	73.28 ± 2.34	67.70 ± 3.28	71.44 ± 0.78
	OUR (GCN)	77.86 ± 1.35	68.76 ± 1.35	76.00 ± 1.12	80.84 ± 1.48	75.84 ± 1.94	83.28 ± 2.01
	OUR (SGC)	75.86 ± 1.59	66.40 ± 2.87	75.36 ± 1.62	77.04 ± 1.91	73.16 ± 4.57	81.14 ± 2.25
	OUR (ARMA)	74.24 ± 1.42	67.00 ± 3.05	73.34 ± 1.19	75.92 ± 3.14	73.66 ± 2.29	77.50 ± 2.75
Attack-3	GCN	75.10 ± 1.38	66.84 ± 1.90	75.94 ± 0.59	79.80 ± 1.03	73.78 ± 2.40	80.02 ± 0.49
	SGC	73.58 ± 2.33	66.34 ± 1.89	75.80 ± 1.01	77.86 ± 2.23	72.44 ± 2.39	79.82 ± 0.57
	ARMA	74.26 ± 2.30	64.44 ± 4.72	73.90 ± 0.64	77.62 ± 2.61	69.62 ± 5.99	77.86 ± 0.43
	OUR (GCN)	78.82 ± 1.01	69.68 ± 1.01	77.72 ± 0.59	82.00 ± 0.98	77.08 ± 0.94	85.34 ± 2.21
	OUR (SGC)	77.66 ± 2.47	69.48 ± 0.95	77.10 ± 0.82	79.14 ± 3.12	78.94 ± 1.23	84.04 ± 1.19
	OUR (ARMA)	78.58 ± 1.46	67.86 ± 1.25	75.78 ± 0.62	80.62 ± 2.47	76.60 ± 1.96	82.04 ± 1.94
Target Model (ARMA)		78.20 ± 1.07	70.53 ± 0.45	77.24 ± 0.92	–	–	–
Attack-0	GCN	59.00 ± 2.25	50.58 ± 5.76	51.50 ± 1.54	59.54 ± 1.46	53.78 ± 5.54	54.74 ± 1.28
	SGC	58.02 ± 1.80	50.76 ± 5.22	51.56 ± 0.86	58.50 ± 2.26	54.12 ± 4.71	54.84 ± 1.23
	ARMA	54.08 ± 1.53	52.04 ± 4.27	51.94 ± 2.15	54.66 ± 1.65	55.14 ± 4.58	54.32 ± 0.54
	OUR (GCN)	66.50 ± 2.56	56.24 ± 3.01	64.76 ± 1.62	67.20 ± 1.43	61.32 ± 3.85	67.00 ± 3.12
	OUR (SGC)	63.24 ± 3.69	57.26 ± 5.35	64.14 ± 1.42	64.74 ± 3.90	61.20 ± 5.41	67.24 ± 2.82
	OUR (ARMA)	61.46 ± 2.39	55.52 ± 4.31	62.84 ± 3.05	62.68 ± 3.26	59.10 ± 3.86	65.10 ± 3.32
Attack-1	GCN	67.50 ± 1.54	62.54 ± 1.87	59.36 ± 2.64	69.06 ± 1.24	64.98 ± 2.95	63.34 ± 2.01
	SGC	66.88 ± 2.29	62.16 ± 1.51	58.94 ± 2.45	68.54 ± 1.26	64.90 ± 2.32	63.06 ± 1.70
	ARMA	64.82 ± 1.08	60.62 ± 0.85	59.08 ± 3.59	67.68 ± 1.33	63.90 ± 1.94	63.08 ± 3.41
	OUR (GCN)	71.54 ± 3.31	64.74 ± 1.17	65.12 ± 3.20	73.22 ± 2.75	72.68 ± 2.41	69.40 ± 3.67
	OUR (SGC)	69.76 ± 2.91	62.66 ± 3.85	66.48 ± 2.09	70.76 ± 4.29	67.94 ± 3.62	70.20 ± 1.44
	OUR (ARMA)	65.78 ± 3.40	63.62 ± 3.74	63.32 ± 1.54	68.30 ± 3.29	71.90 ± 4.45	68.20 ± 1.11
Attack-2	GCN	68.58 ± 2.07	65.64 ± 3.01	68.18 ± 2.09	71.60 ± 1.26	69.70 ± 4.15	72.96 ± 0.75
	SGC	67.82 ± 1.96	66.00 ± 1.84	67.08 ± 2.56	70.84 ± 1.98	70.58 ± 2.62	72.28 ± 1.67
	ARMA	67.72 ± 0.43	66.00 ± 1.57	65.76 ± 1.90	71.00 ± 2.30	70.66 ± 2.04	70.98 ± 1.17
	OUR (GCN)	74.38 ± 2.92	68.88 ± 1.13	72.62 ± 1.97	75.70 ± 3.79	75.50 ± 1.76	77.80 ± 2.41
	OUR (SGC)	73.04 ± 1.49	66.14 ± 3.47	71.94 ± 2.65	75.36 ± 2.49	71.40 ± 5.91	76.08 ± 3.54
	OUR (ARMA)	70.70 ± 2.31	66.56 ± 0.65	68.52 ± 1.29	71.68 ± 1.72	74.52 ± 1.83	75.00 ± 1.97
Attack-3	GCN	72.36 ± 1.84	68.16 ± 1.49	72.74 ± 1.31	75.72 ± 1.34	74.70 ± 1.97	78.58 ± 1.33
	SGC	70.80 ± 2.01	67.36 ± 1.35	72.56 ± 1.51	73.76 ± 2.69	73.86 ± 1.56	78.54 ± 1.53
	ARMA	71.44 ± 2.94	67.24 ± 0.92	71.04 ± 1.17	75.86 ± 3.37	73.62 ± 1.64	77.66 ± 1.20
	OUR (GCN)	77.68 ± 1.28	69.24 ± 0.73	74.58 ± 2.66	78.90 ± 2.36	76.24 ± 1.78	79.34 ± 1.34
	OUR (SGC)	77.52 ± 2.02	67.46 ± 1.74	74.98 ± 1.19	78.10 ± 2.42	75.04 ± 1.97	78.96 ± 1.16
	OUR (ARMA)	75.32 ± 1.86	67.38 ± 0.98	72.18 ± 0.73	78.30 ± 0.96	75.54 ± 0.73	79.66 ± 0.95

As can be seen in the table, our approach consistently improved performance regardless of the surrogate model's architecture. Notably, the results show a marked improvement in model performance as the number of queryable nodes increases, i.e., as we progressed from Attack-0 through to Attack-3.

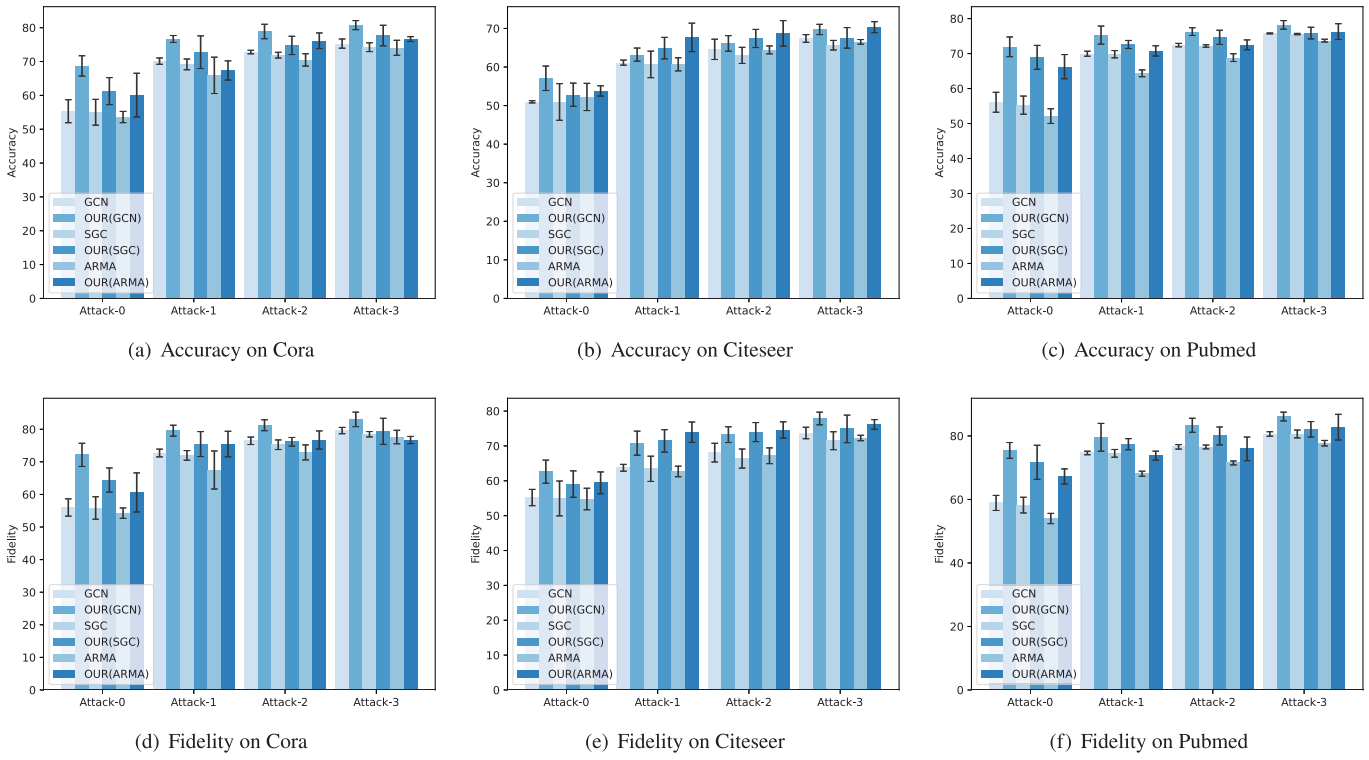
To visually represent this generality, Fig. 6 presents a histogram comparison between our proposed method and a typical existing attack under three different surrogate models across a range of attacks and datasets. The histograms clearly show that, with the same configuration, our proposed method consistently outperforms the original in both accuracy and fidelity. This underscores the applicability of our method to different GNN architectures, establishing it as a generalized GNN model extraction attack method.

6.4.2. Different target model architectures

We also conducted experiments with various target model architectures to validate the effectiveness of our approach. In Table 6, Target ARMA (SGC) and Target Model (ARMA) denotes where we used SGC and ARMA networks to form the target model's architecture.

From the results, we can see that the proposed method results in both a higher accuracy and a higher fidelity. More specifically, in Attack-0, our approach yields an improvement of up to 15% with SGC as the target model. With ARMA as the target, our method similarly boosts accuracy and fidelity by 13%. Additionally, it is noteworthy that the performance of each attack method under different architectures is relatively consistent, and the attack performance of each method improves as the number of queries increases. Moreover, regardless of the target model architecture, our method consistently surpasses the original method in all datasets and attack scenarios.

Furthermore, in some scenarios, the same attack exhibits significant differences under different target models, reflecting the performance disparities of the adopted models. For instance, under Cora, the OUR(GCN) method is 3% more accurate with GCN as the target model



(a) Accuracy on Cora (b) Accuracy on Citeseer (c) Accuracy on Pubmed

(d) Fidelity on Cora (e) Fidelity on Citeseer (f) Fidelity on Pubmed

Fig. 6. Comparison of our proposed method with existing methods.

compared to ARMA as the target model. This difference is attributed to GCN having an accuracy of 81.04% for node classification in Cora, while ARMA has only 78.2% accuracy for node classification in the same dataset. The varying accuracies of each model contribute to notable differences in the performance of our method under different models.

6.5. Comparison with other attack methods

To demonstrate the superiority of our proposed method, we conducted a comparative analysis against recent studies [18,19] that explore generic GNN model extraction attacks. These approaches investigate model extraction attacks with varying levels of attacker knowledge.

In [18], the authors explore model extraction attacks under varying levels of attacker knowledge. They impose constraints on the attacker's knowledge and examine how these constraints affect the effectiveness of model extraction attacks. However, it should be emphasized that their study is focused on assessing the impact of limited knowledge rather than introducing new model extraction attack techniques. In Table 5, the outcomes of the original models under Attack-0 to Attack-3 illustrate their investigation into scenario-based model extraction attacks. As previously discussed, our revised approach is superior. However, what is important to observe is that, while they propose a scenario involving model extraction with partial nodes, they do not account for the possibility that response labels might be erroneous.

In [19], the authors impose restrictions on the response content of the target model. To enable a comparative analysis with their work, we conducted experiments using posterior probabilities as response outcomes, and the results are detailed in Table 7. Even under this constraint, our model outperforms in model extraction across various datasets and scenarios. Notably, on the Cora dataset, our proposed method achieves an accuracy of 80%, closely matching the target model's accuracy of 81.04%. Furthermore, our method shows enhanced performance with model extraction attacks given limited queries and under the condition that some of the labels are incorrect. In Attack-0, our method resulted in a significant improvement to model accuracy, even when using posterior probabilities. This is best demonstrated visually using the bar chart comparison of Attack-0 in Fig. 7. The figure clearly shows the substantial improvements in accuracy and fidelity with our proposed method.

Here, it is important to clarify that our method using posterior probabilities does not introduce incorrect labels. Consequently, as the number of query nodes increases, the performance improvement of our approach gradually decreases, due to the reduction in the erroneous information to correct. This experimental result further demonstrates that our method can significantly enhance the success rate of model extraction attacks while minimizing the number of queries.

6.6. Evaluations with large graph datasets

To further assess the practical applicability of the proposed method, we conducted additional experiments on larger graph datasets, namely the Physics and Ogbn-arxiv datasets. The results of these experiments are depicted in Fig. 8.

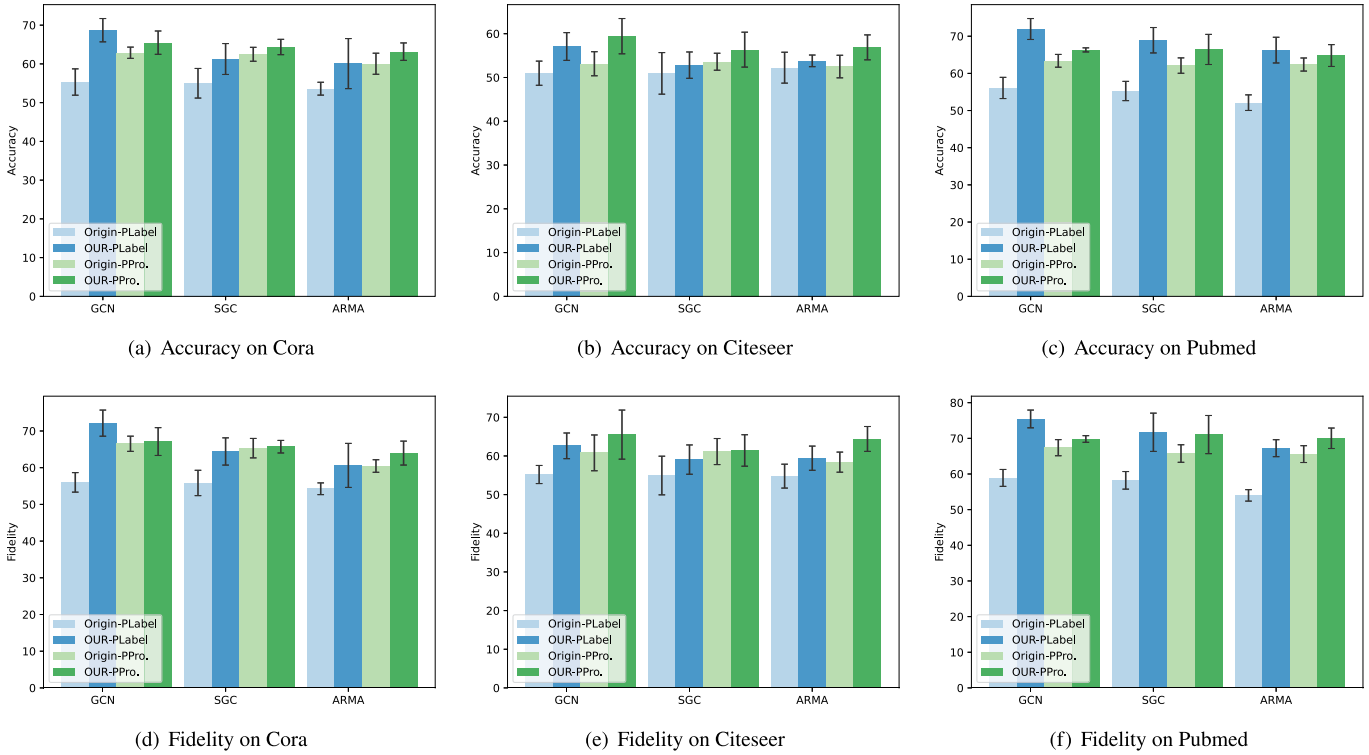
As shown in the figure, the method proposed in this paper significantly enhances the performance of the model extraction attack on large graphs. For instance, in Attack-0 on Physics, our method improves the attack accuracy from the original 82.88% to 93.31%, marking an improvement of over 10 percentage points. Similar enhancements are observed across all other attack settings. On Ogbn-arxiv, although the accuracy is lower due to the dataset's 40-class classification, our method still achieves over 60% fidelity for each attack setting. This underscores the robust practical applicability of our approach.

7. Discussions

GNNs have been extensively studied and applied across a variety of tasks involving graph data, including graph-level, node-level, and link-level analyses. It is important to note that our work is primarily concerned with node-level classification tasks, and the effectiveness of our proposed model extraction technique has been evaluated within

Table 7
Results of each attack method when the target model response content is posterior probabilistic.

Metrics		Accuracy			Fidelity		
Dataset		Cora	Citeseer	Pubmed	Cora	Citeseer	Pubmed
Target Model (Posterior Probability)		81.04 ± 0.34	71.10 ± 0.29	78.68 ± 0.87	–	–	–
Attack-0	GCN	62.88 ± 1.45	53.14 ± 2.75	63.38 ± 1.72	66.52 ± 2.07	60.80 ± 4.63	67.38 ± 2.26
	SGC	62.48 ± 1.80	53.60 ± 1.94	62.10 ± 2.07	65.32 ± 2.65	61.14 ± 3.37	65.74 ± 2.46
	ARMA	60.04 ± 2.72	52.50 ± 2.58	62.38 ± 1.76	60.46 ± 1.71	58.40 ± 2.60	65.58 ± 2.36
	OUR (GCN)	65.48 ± 3.01	59.44 ± 4.03	66.32 ± 0.55	67.12 ± 3.79	65.52 ± 6.35	69.82 ± 0.93
	OUR (SGC)	64.36 ± 1.98	56.36 ± 3.99	66.44 ± 4.05	65.72 ± 1.71	61.42 ± 4.07	71.06 ± 5.35
	OUR (ARMA)	63.16 ± 2.25	56.88 ± 2.85	64.80 ± 2.94	63.98 ± 3.26	64.40 ± 3.22	70.02 ± 2.87
Attack-1	GCN	69.66 ± 0.98	66.76 ± 2.01	72.50 ± 1.08	75.08 ± 0.76	73.12 ± 3.10	77.64 ± 0.81
	SGC	66.82 ± 1.24	65.08 ± 2.45	72.12 ± 0.90	71.98 ± 1.10	71.84 ± 2.84	77.22 ± 1.79
	ARMA	66.54 ± 2.27	65.60 ± 1.36	69.56 ± 0.58	69.46 ± 2.70	71.78 ± 0.57	73.62 ± 0.70
	OUR (GCN)	75.46 ± 3.01	67.76 ± 2.28	72.80 ± 2.31	79.84 ± 3.68	75.10 ± 3.44	80.04 ± 2.09
	OUR (SGC)	70.76 ± 4.46	65.12 ± 1.96	73.10 ± 3.33	74.30 ± 4.03	73.74 ± 1.08	79.06 ± 3.98
	OUR (ARMA)	68.92 ± 2.08	66.14 ± 2.20	70.58 ± 1.86	71.20 ± 2.58	75.74 ± 2.41	76.74 ± 3.05
Attack-2	GCN	76.84 ± 0.74	68.16 ± 1.67	76.18 ± 0.77	83.08 ± 0.93	76.98 ± 1.63	82.52 ± 0.38
	SGC	75.10 ± 0.97	67.02 ± 1.18	75.52 ± 1.18	80.88 ± 0.73	75.62 ± 2.25	82.18 ± 0.85
	ARMA	72.50 ± 3.05	67.72 ± 1.70	72.86 ± 0.51	76.50 ± 3.81	74.66 ± 1.36	77.34 ± 1.04
	OUR (GCN)	77.62 ± 1.58	69.62 ± 0.66	76.46 ± 2.21	83.42 ± 1.03	77.90 ± 3.24	84.06 ± 2.08
	OUR (SGC)	75.36 ± 1.61	68.32 ± 1.87	75.70 ± 1.84	81.02 ± 0.79	77.48 ± 2.02	82.12 ± 1.47
	OUR (ARMA)	75.40 ± 4.74	68.70 ± 1.61	73.18 ± 0.54	77.82 ± 5.48	76.12 ± 3.58	78.78 ± 1.40
Attack-3	GCN	78.00 ± 0.96	69.92 ± 1.15	77.04 ± 1.21	84.56 ± 0.74	79.72 ± 1.28	86.18 ± 0.54
	SGC	76.78 ± 1.04	69.36 ± 0.79	76.62 ± 1.31	83.02 ± 1.17	79.48 ± 1.23	85.40 ± 1.57
	ARMA	76.66 ± 2.04	69.00 ± 1.41	74.96 ± 0.83	81.32 ± 2.91	78.24 ± 1.35	81.84 ± 0.72
	OUR (GCN)	80.18 ± 0.70	70.98 ± 0.93	78.32 ± 0.44	85.52 ± 1.63	80.76 ± 1.11	86.32 ± 0.74
	OUR (SGC)	78.26 ± 1.78	69.48 ± 1.75	77.46 ± 1.57	83.40 ± 1.38	80.06 ± 1.03	86.00 ± 0.39
	OUR (ARMA)	78.42 ± 1.52	69.30 ± 0.98	75.60 ± 1.35	81.94 ± 1.05	79.26 ± 0.65	82.84 ± 1.34



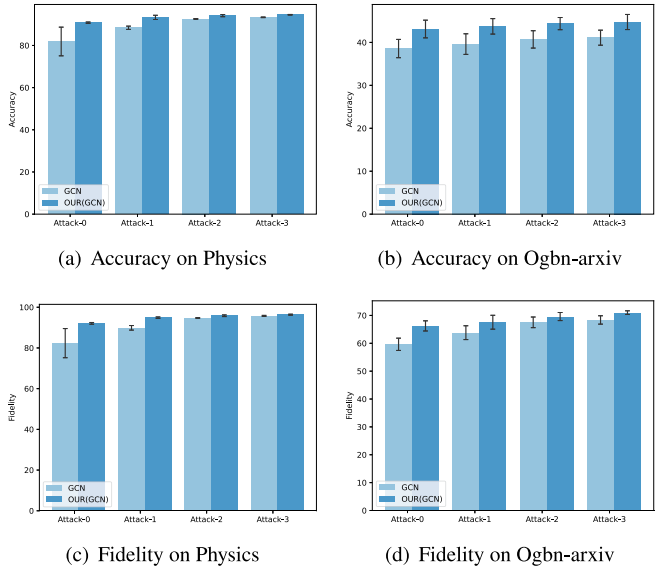
(a) Accuracy on Cora (b) Accuracy on Citeseer (c) Accuracy on Pubmed

(d) Fidelity on Cora (e) Fidelity on Citeseer (f) Fidelity on Pubmed

Fig. 7. Comparison of results between our method and the existing method with the posterior probabilistic response content of the target model, along with an additional comparison of the attack methods for different response contents of the target model. *PLabel* indicates that the response content is predicted labeled, and *PPro.* indicates that the response content is posterior probabilistic.

this specific context. We have not explored the applicability of our approach to graph-level or link-level tasks, and it is likely that some form of adaptation or modification would be required to effectively conduct model extraction attacks in these distinct settings. Future research should aim to adapt and refine our approach to meet the unique challenges and requirements of graph-level and link-level tasks.

At present, two primary learning paradigms for GNNs exist: transductive learning and inductive learning. Shen et al. [19] focus on



(a) Accuracy on Physics (b) Accuracy on Ogbn-arxiv

(c) Fidelity on Physics (d) Fidelity on Ogbn-arxiv

Fig. 8. Comparison on the larger Physics and Ogbn-arxiv datasets.

inductive GNNs in their research and, similarly, this study specifically experimented with transductive GNN skeletons; we did not explore model extraction on the backbone of inductive GNNs. Consequently, additional research is warranted to assess the effectiveness of our extraction approach with inductive GNNs. Investigating the unique challenges and characteristics of inductive GNNs is crucial to developing tailored techniques for successful model extraction attacks in this context.

Furthermore, in this paper, our focus is solely on enhancing the information transfer of GNN models by adding potential edges, thereby enabling model extraction attacks with limited predicted labels. However, the idea of deleting edges to optimize the model extraction attack process is an interesting direction that we intend to explore in future research.

8. Conclusion

In this paper, we investigated model extraction attacks on graph neural networks within a restricted black-box setting. Comparing with existing model extraction attacks, this setting involves more realistic scenarios where the target model may generate incorrect labels and has a limited access to the target model. We propose a novel extraction methodology that utilizes the graph's topological information and enables efficient information propagation. The goal is to mitigate the impact of incorrect labels and enhance the efficiency of model extraction. Importantly, our approach shows superior performance even with a limited number of queries and without requiring prior knowledge of the target model's structure. In addition, we provide a theoretical analysis and empirical evaluation to support the effectiveness of the presented approach. Our experimental results show that our method outperforms the benchmark approaches in terms of attack performance. Furthermore, we explore the effectiveness of our approach across various surrogate and target models. Overall, our methodology surpasses existing methods and achieves state-of-the-art results.

CRediT authorship contribution statement

Faqian Guan: Writing – original draft, Conceptualization. **Tianqing Zhu:** Methodology, Conceptualization. **Hanjin Tong:** Software. **Wanlei Zhou:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] Y. Wu, D. Lian, Y. Xu, L. Wu, E. Chen, Graph convolutional networks with Markov random field reasoning for social spammer detection, in: The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI, 2020, pp. 1054–1061.

[2] W.L. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, NeurIPS, 2017, pp. 1024–1034.

[3] H. Wang, W. Yang, W. Wang, D. Man, J. Lv, A novel cross-network embedding for anchor link prediction with social adversarial attacks, ACM Trans. Priv. Secur. 26 (1) (2023) 7:1–7:32.

[4] W. Fan, Y. Ma, Q. Li, Y. He, Y.E. Zhao, J. Tang, D. Yin, Graph neural networks for social recommendation, in: The World Wide Web Conference, WWW, 2019, pp. 417–426.

[5] Y. Ge, J. Ma, L. Zhang, X. Li, H. Lu, Trustworthiness-aware knowledge graph representation for recommendation, Knowl.-Based Syst. 278 (2023) 110865.

[6] C. Chang, J. Zhou, Y. Weng, X. Zeng, Z. Wu, C. Wang, Y. Tang, KGTN: Knowledge graph transformer network for explainable multi-category item recommendation, Knowl.-Based Syst. 278 (2023) 110854.

[7] F. Scarselli, M. Gori, A.C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, IEEE Trans. Neural Netw. 20 (1) (2009) 61–80.

[8] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017, in: Proceedings of Machine Learning Research, vol. 70, PMLR, 2017, pp. 1263–1272.

[9] J. Klicpera, A. Bojchevski, S. Günnemann, Predict then propagate: Graph neural networks meet personalized PageRank, in: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, la, USA, May 6-9, 2019, 2019, OpenReview.net.

[10] S. Zhu, S. Pan, C. Zhou, J. Wu, Y. Cao, B. Wang, Graph geometry interaction learning, in: Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, Virtual, 2020.

[11] T. Wu, X. Wang, S. Qiao, X. Xian, Y. Liu, L. Zhang, Small perturbations are enough: Adversarial attacks on time series prediction, Inform. Sci. 587 (2022) 794–812.

[12] T. Wu, N. Yang, L. Chen, X. Xiao, X. Xian, J. Liu, S. Qiao, C. Cui, ERGCN: Data enhancement-based robust graph convolutional network against adversarial attacks, Inform. Sci. 617 (2022) 234–253.

[13] F. Guan, T. Zhu, W. Zhou, K.R. Choo, Graph neural networks: a survey on the links between privacy and security, Artif. Intell. Rev. 57 (2) (2024) 40.

[14] F. Tramèr, F. Zhang, A. Juels, M.K. Reiter, T. Ristenpart, Stealing machine learning models via prediction APIs, in: 25th USENIX Security Symposium, USENIX, 2016, pp. 601–618.

[15] R. Shokri, M. Stronati, C. Song, V. Shmatikov, Membership inference attacks against machine learning models, in: 2017 IEEE Symposium on Security and Privacy, SP, 2017, pp. 3–18.

[16] V. Torra, G. Navarro-Arribas, Attribute disclosure risk for k-anonymity: the case of numerical data, Int. J. Inf. Sec. 22 (6) (2023) 2015–2024.

[17] D. DeFazio, A. Ramesh, Adversarial model extraction on graph neural networks, 2019, CoRR. arXiv:1912.07721.

[18] B. Wu, X. Yang, S. Pan, X. Yuan, Model extraction attacks on graph neural networks: Taxonomy and realisation, in: ASIA CCS '22: ACM Asia Conference on Computer and Communications Security, 2022, pp. 337–350.

[19] Y. Shen, X. He, Y. Han, Y. Zhang, Model stealing attacks against inductive graph neural networks, in: 43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022, IEEE, 2022, pp. 1175–1192.

[20] L. Sun, Y. Dou, C.J. Yang, K. Zhang, J. Wang, P.S. Yu, L. He, B. Li, Adversarial attack and defense on graph data: A survey, IEEE Trans. Knowl. Data Eng. 35 (8) (2023) 7693–7711.

[21] A. Yan, H. Yan, L. Hu, X. Liu, T. Huang, Holistic implicit factor evaluation of model extraction attacks, IEEE Trans. Depend. Secur. Comput. 20 (6) (2023) 4678–4689.
[22] A. Yan, R. Hou, H. Yan, X. Liu, Explanation-based data-free model extraction attacks, World Wide Web (WWW) 26 (5) (2023) 3081–3092.
[23] A. Yan, T. Huang, L. Ke, X. Liu, Q. Chen, C. Dong, Explanation leaks: Explanation-guided model extraction attacks, Inform. Sci. 632 (2023) 269–284.
[24] Y. Chen, R. Guan, X. Gong, J. Dong, M. Xue, D-DAE: defense-penetrating model extraction attacks, in: 44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023, IEEE, 2023, pp. 382–399.
[25] D. Li, Di Liu, Y. Guo, Y. Ren, J. Su, J. Liu, Defending against model extraction attacks with physical unclonable function, Inform. Sci. 628 (2023) 196–207.
[26] G. Liu, T. Xu, R. Zhang, Z. Wang, C. Wang, L. Liu, Gradient-leaks: Enabling black-box membership inference attacks against machine learning models, IEEE Trans. Inf. Forensics Secur. 19 (2024) 427–440.
[27] Y. Liu, P. Jiang, L. Zhu, Subject-level membership inference attack via data augmentation and model discrepancy, IEEE Trans. Inf. Forensics Secur. 18 (2023) 5848–5859.
[28] K. Wang, J. Wu, T. Zhu, W. Ren, Y. Hong, Defense against membership inference attack in graph neural networks through graph perturbation, Int. J. Inf. Sec. 22 (2) (2023) 497–509.
[29] M. Fredrikson, E. Lantz, S. Jha, S.M. Lin, D. Page, T. Ristenpart, Privacy in pharmacogenetics: An end-to-end case study of personalized warfarin dosing, in: Proceedings of the 23rd USENIX Security Symposium, 2014, pp. 17–32.
[30] Y. Hu, Y. Zhang, X. Gao, D. Gong, X. Song, Y. Guo, J. Wang, A federated feature selection algorithm based on particle swarm optimization under privacy protection, Knowl.-Based Syst. 260 (2023) 110122.
[31] Z. Zhang, Q. Liu, Z. Huang, H. Wang, C. Lu, C. Liu, E. Chen, GraphMI: Extracting private graph data from graph neural networks, in: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI, 2021, pp. 3749–3755.
[32] Z. Zhang, M. Chen, M. Backes, Y. Shen, Y. Zhang, Inference attacks against graph neural networks, in: Proc. USENIX Security, 2022.
[33] E. Dai, C. Aggarwal, S. Wang, NRGNN: Learning a label noise resistant graph neural network on sparsely and noisily labeled graphs, in: KDD '21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, 2021, pp. 227–236.
[34] X. He, J. Jia, M. Backes, N.Z. Gong, Y. Zhang, Stealing links from graph neural networks, in: M.D. Bailey, R. Greenstadt (Eds.), 30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021, USENIX Association, 2021, pp. 2669–2686.
[35] L. Wang, J. Luo, S. Deng, X. Guo, RoCS: Knowledge graph embedding based on joint cosine similarity, Electronics (ISSN: 2079-9292) 13 (1) (2024).
[36] Y. Takano, Y. Iijima, K. Kobayashi, H. Sakuta, H. Sakaji, M. Kohana, A. Kobayashi, Improving document similarity calculation using cosine-similarity graphs, in: L. Barolli, M. Takizawa, F. Xhafa, T. Enokido (Eds.), Advanced Information Networking and Applications - Proceedings of the 33rd International Conference on Advanced Information Networking and Applications, AINA 2019, Matsue, Japan, March 27-29, 2019, in: Advances in Intelligent Systems and Computing, vol. 926, Springer, 2019, pp. 512–522.
[37] T. Mikolov, I. Sutskever, K. Chen, G.S. Corrado, J. Dean, Distributed representations of words and phrases and their compositionality, in: C.J.C. Burges, L. Bottou, Z. Ghahramani, K.Q. Weinberger (Eds.), Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a Meeting Held December 5-8, 2013, Lake Tahoe, Nevada, United States, 2013, pp. 3111–3119.
[38] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: Y. Bengio, Y. LeCun (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings, 2015.
[39] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings, 2017, OpenReview.net.
[40] F. Wu, A.H.S. Jr., T. Zhang, C. Fifty, T. Yu, K.Q. Weinberger, Simplifying graph convolutional networks, in: K. Chaudhuri, R. Salakhutdinov (Eds.), Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA, in: Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 6861–6871.
[41] F.M. Bianchi, D. Grattarola, L. Livi, C. Alippi, Graph neural networks with convolutional ARMA filters, IEEE Trans. Pattern Anal. Mach. Intell. 44 (7) (2022) 3496–3507.
[42] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, in: Relational Representation Learning Workshop, NeurIPS 2018, 2018.
[43] Q. Zheng, X. Zou, Y. Dong, Y. Cen, D. Yin, J. Xu, Y. Yang, J. Tang, Graph robustness benchmark: Benchmarking the adversarial robustness of graph machine learning, in: Neural Information Processing Systems Track on Datasets and Benchmarks 2021, 2021.
[44] C. Gao, B. Li, Y. Wang, W. Chen, L. Zhang, Tenet: A neural network model extraction attack in multi-core architecture, in: Y. Chen, V.V. Zhirnov, A. Sasan, I. Savidis (Eds.), GLSVLSI '21: Great Lakes Symposium on VLSI 2021, Virtual Event, USA, June 22-25, 2021, ACM, 2021, pp. 21–26.
[45] T. Orekondy, B. Schiele, M. Fritz, Knockoff nets: Stealing functionality of black-box models, in: IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019, Computer Vision Foundation / IEEE, 2019, pp. 4954–4963.